at runtime. To see an example of this technique, look back at your original refactoring problem.

Refactoring the Computer Class

Recall the code that pulled you and Bill into this dynamic discussion:

methods/computer/duplicated.rb

```
class Computer
  def initialize(computer_id, data_source)
    @id = computer_id
    @data_source = data_source
  end

  def mouse
    info = @data_source.get_mouse_info(@id)
    price = @data_source.get_mouse_price(@id)
    result = "Mouse: #{info} ($#{price})"
    return "* #{result}" if price >= 100
    result
  end

  def cpu
    info = @data_source.get_cpu_info(@id)
    price = @data_source.get_cpu_price(@id)
    result = "Cpu: #{info} ($#{price})"
    return "* #{result}" if price >= 100
    result
  end

  def keyboard
    info = @data_source.get_keyboard_info(@id)
    price = @data_source.get_keyboard_price(@id)
    result = "Keyboard: #{info} ($#{price})"
    return "* #{result}" if price >= 100
    result
  end

  # ...
end
```

In the previous pages you learned how to use Module#define_method in place of the def keyword to define a method, and how to use send in place of the dot notation to call a method. Now you can use these spells to refactor the Computer class. It's time to remove some duplication.

Step 1: Adding Dynamic Dispatches

You and Bill start by extracting the duplicated code into its own message-sending method: